GEBZE TECHNICAL UNIVERSITY

Computer Engineering Department

CSE 396 Module Documentation

Çılgın Kurye

Module M4: Traffic and AI

Company: Board&Byte **Group:** Group-8 **Date:** 6 November 2026

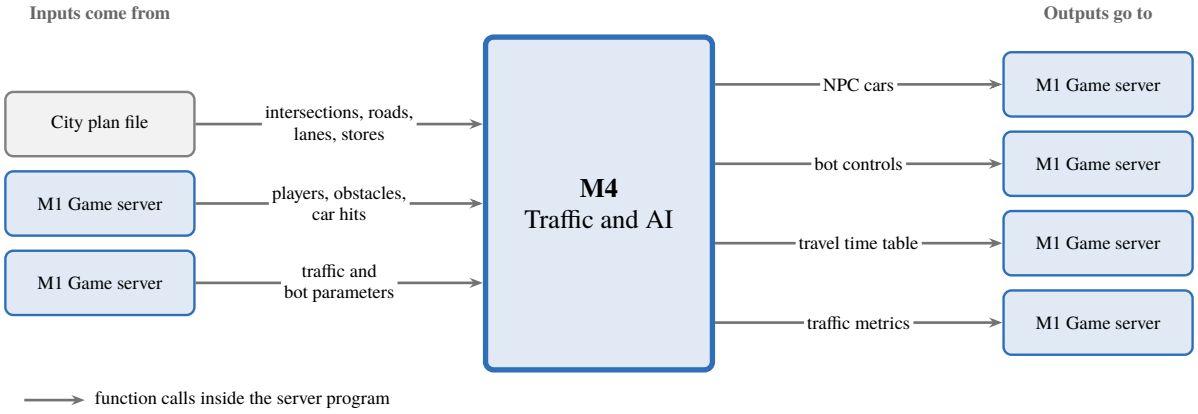
Name	Student No.	Name	Student No.
Hasan Alperen Elmas	1801042604	Batuhan Yanar	220104004059
Muhib Taha Boy	230104004165		

1. Purpose

M4 brings the city to life. It finds routes and travel times on the road network. It moves the NPC cars that make the traffic. It also drives the AI bots that fill the empty places in a game. A bot makes the same control message that a phone sends, so for M1 a bot and a human player look the same.

Runs on	Technology	Requirements
Main computer inside the game server. M1 runs it once in every tick	TypeScript, our own shortest path search (Dijkstra), a car following model for NPC cars, a state machine and path following for bots, seeded random numbers	R1 R2 R4 R6 R11

M4 talks only to M1, inside the same program. The figure shows what M4 gets from M1 in every tick and what it gives back.



2. Requirements covered

M4 takes part in five requirements. The last column says what will work when M4 is finished.

ID	Requirement	What M4 does for it	When M4 is done
R1	Multiplayer game	Bots take the empty places and play like players	A game can start with only one human player
R2	Delivery cycle	Travel time table from each store, which M1 uses for fair near and far points	No team gets easier delivery points than another
R4	Traffic	NPC cars make real traffic on the roads	The city is full of cars that never lock up
R6	Obstacles	Heavy traffic adds slow cars. An accident closes a lane. Road work closes the road for routes and cars	Cars and bots go around every obstacle
R11	Engineering requirements	Talks only to M1 with plain data, tested with its own test program	M4 can be built and tested without the other modules

3. Inputs and outputs

M1 gives the inputs at the start of each tick and takes the outputs before M3 moves the motorcycles.

Direction	Data	From / to	Content
In	City plan	Shared file	Intersections, roads, lanes, stores, possible delivery points
In	Game data	M1	Every tick: players, obstacles, car hit events
In	Parameters	M6, through M1	Number of NPC cars, traffic effects, bot skill
Out	NPC cars	M1, then M3, M5, M6	Position, direction, speed, lane, size
Out	Bot controls	M1, then M3	The same record as a phone: throttle, brake, steer
Out	Travel times	M1	Seconds from each team store to every possible delivery point
Out	Traffic metrics	M1, then M6	Number of NPC cars, mean NPC speed, bot route changes

4. Main components

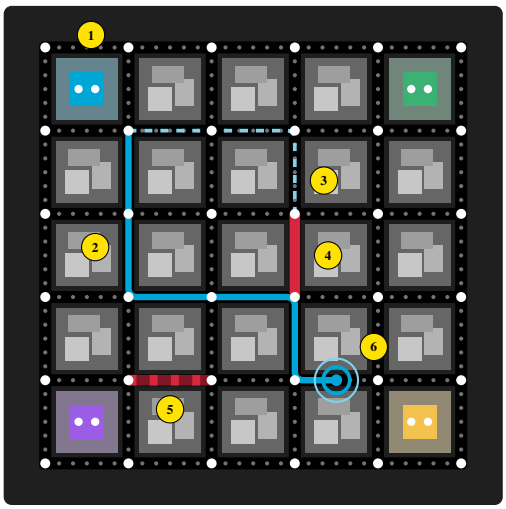
M4 has five parts. The route planner is used by both the NPC cars and the bots.

Component	Takes	Gives
Road graph	City plan	36 intersections and 60 roads in both directions, with lanes and turn costs
Route planner	Road graph, obstacles, traffic, parameters	Fastest routes and the travel time table, computed again when a cost changes
NPC traffic	Routes, players, car hits, number of cars	Moved, added and removed NPC cars
Bot drivers	Routes, the bot's motorcycle, game data	One control record for each bot in every tick
Traffic metrics	Cars and bots	Values for the admin graphs

5. How it works

The road graph and routes:

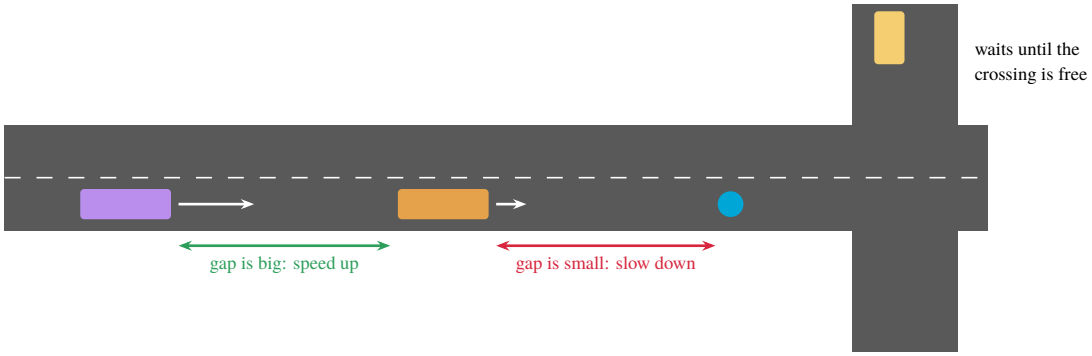
The city has 6 roads in each direction. Where two roads cross, there is an intersection, so the graph has 36 intersections. Between them there are 60 road segments, and cars can drive both ways. Each segment has a cost: the expected time to drive it. Turning left or making a U turn adds a small extra cost. Obstacles change the costs. Heavy traffic and an accident make a segment slower, and road work closes it. M4 uses Dijkstra's shortest path search to find the fastest routes. The graph is small, so one search is very fast. M4 computes the routes again whenever a cost changes: when the obstacle list changes, when the traffic on a road gets much heavier or lighter, or when the admin changes a route parameter. From the results it makes the travel time table: the time from each of the four stores to every possible delivery point. M1 uses this table to choose fair near and far points.



1 Start: the team store
2 Fastest route, found by Dijkstra
3 Shorter road, but closed
4 Road work: cost is infinite
5 Heavy traffic: cost is higher
6 Delivery point
◦ Intersection, a node of the graph

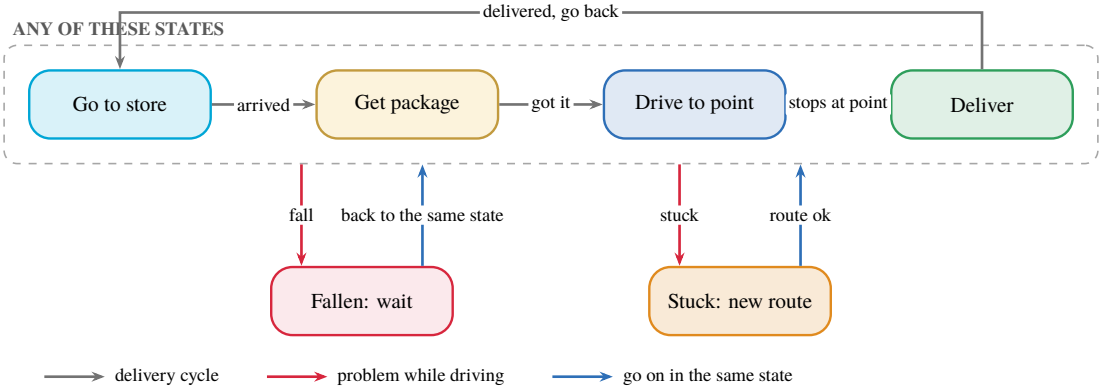
NPC traffic:

Each NPC car follows a lane and picks a new turn at each intersection. A car keeps a safe gap behind the car or the motorcycle in front of it. If the gap gets small, it slows down smoothly, and it speeds up again when the road is free. A car enters an intersection only when the intersection is free, so cars do not drive through each other. New cars never appear close to a player. If a car is stuck for too long, M4 removes it and adds it again somewhere else, so the traffic never locks up. The admin sets the number of cars, and heavy traffic obstacles add extra slow cars on their road.



Bots:

Every bot has a small state machine that follows the delivery cycle, as the figure below shows. The bot drives to the store, gets the package, drives to the delivery point, stops there so the package is delivered, and then goes back. On the way it follows the route from the planner. It steers toward a point a little ahead on the path, so the motion looks smooth. A bot can fall or get stuck in any of these four states. If it falls, it waits for the penalty time and then goes on in the same state. If it is stuck, it asks for a new route and goes on in the same state. One "bot skill" value sets the top speed, the reaction time and some steering noise, so the admin can make bots weaker or stronger.



6. Testing

M4 is tested alone with a small test program that plays the role of M1. It gives the city plan, fixed player positions and a list of test obstacles, and it draws the city from above. We check routes with and without closed roads, equal travel times for all teams in a city that is the same on all sides, new routes when the traffic changes, many cars in a long run without a traffic lock, bots that finish their deliveries, and the same result with the same seed.